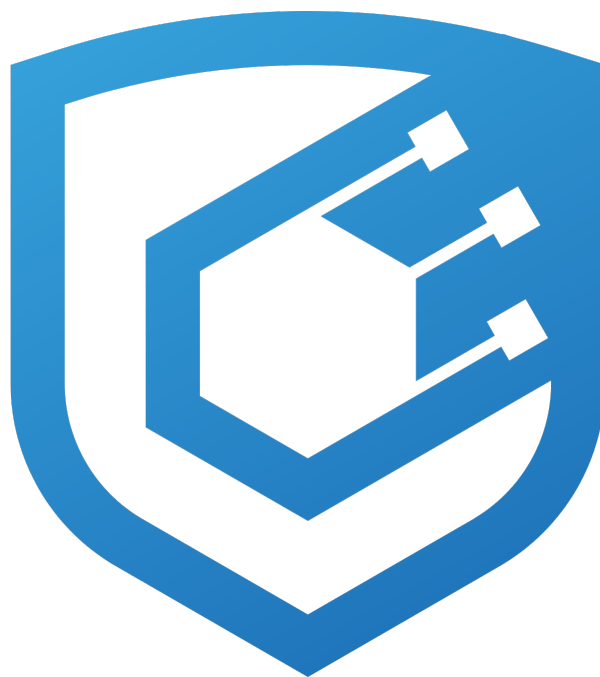




XAUM EVM Audit Report

Prepared by [Cyfrin](#)

Version 2.0

Lead Auditors

[Dacian](#)

[Stalin](#)

January 26, 2026

Contents

1	About Cyfrin	2
2	Disclaimer	2
3	Risk Classification	2
4	Protocol Summary	2
5	Audit Scope	2
6	Executive Summary	2
7	Findings	5
7.1	Low Risk	5
7.1.1	Existing MToken::revoker can prevent owner from setting new revoker	5
7.1.2	MToken::mintTo, redeem, ccReceiveToken, pack, unpack never enforce the blocklist	5
7.1.3	MToken::transfer, transferFrom, multiTransfer allow transferring to blocked recipients	5
7.1.4	Using tx.origin allows possibility for third-party contract to shuffle cross-chain mint budgets without operator consent	5
7.1.5	Tokens can be sent cross-chain to blocked users on non-EVM chains	6
7.2	Informational	7
7.2.1	Use named imports	7
7.2.2	Use capital case for constants	7
7.2.3	Use Ownable2StepUpgradeable instead of OwnableUpgradeable	7
7.2.4	Use _checkZeroAddress in DelayedUpgradeable::requestUpgradeToAndCall	8
7.2.5	Emit missing events in state-changing functions	8
7.2.6	In Solidity don't initialize to default values	8
7.2.7	No restriction on callers to BullionMinter::requestToMint, requestToRedeem allows off-chain parameter checks to be bypassed	8
7.2.8	Owner can rug-pull user tokens by front-running calls to BullionMinter::requestToMint, requestToRedeem to change poolAccountA, poolAccountB	8
7.2.9	Remove unused variables	9
7.2.10	MToken::mintTo nonce never enforced on-chain	9
7.2.11	Use named mappings to explicitly denote the purpose of keys and values	9
7.2.12	Remove obsolete return statement when already using named return variables	9
7.2.13	Fallback oracle contains no staleness check	9
7.2.14	Consider using SafeCast instead of unsafe downcasting	10
7.2.15	No way to specifically pause CCIP send transactions as exists for LayerZero send transactions	10
7.2.16	Cross-chain transfers can cause totalSupply to be greater than mintBudget on the destination chain	11
7.3	Gas Optimization	12
7.3.1	Cache storage to prevent identical storage reads	12
7.3.2	Use calldata instead of memory for read-only inputs	12
7.3.3	Perform dstAddrLen revert checks first in MTokenMessengerLZ::lzSendTokenToChain	12

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the solidity implementation of the contracts.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

Matrixdock is a Real World Asset (RWA) Tokenization protocol which has tokenized US Treasuries as STBT and physical gold as XAUm.

Currently Matrixdock uses both Chainlink CCIP & LayerZero to allow token holders to bridge their tokens between EVM-compatible blockchains. The purpose of this audit is to perform a security review of an upgrade designed to allow cross-chain token bridging to non-EVM blockchains such as Solana and Sui.

5 Audit Scope

The audit scope was limited to:

```
xaum-evm/contracts/BullionMinter.sol
xaum-evm/contracts/Delayable.sol
xaum-evm/contracts/DelayedUpgradeable.sol
xaum-evm/contracts/MToken.sol
xaum-evm/contracts/MTokenMessenger.sol
xaum-evm/contracts/MTokenMessengerBaseUpgradeable.sol
xaum-evm/contracts/MTokenMessengerLZ.sol
```

6 Executive Summary

Over the course of 4 days, the Cyfrin team conducted an audit on the [XAUM EVM](#) smart contracts provided by [Matrixdock](#). In this period, a total of 24 issues were found.

The findings consist of 5 Low severity issues with the remainder being informational and gas optimizations.

Summary

Project Name	XAUM EVM
Repository	RWA-Contracts
Commit	0fb0eb676fb9...
Fix Commit	25ca660397a3...
Audit Timeline	Jan 19th - Jan 22nd, 2026
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	5
Informational	16
Gas Optimizations	3
Total Issues	24

Summary of Findings

[L-1] Existing MToken::revoker can prevent owner from setting new revoker	Resolved
[L-2] MToken::mintTo, redeem, ccReceiveToken, pack, unpack never enforce the blacklist	Acknowledged
[L-3] MToken::transfer, transferFrom, multiTransfer allow transferring to blocked recipients	Acknowledged
[L-4] Using tx.origin allows possibility for third-party contract to shuffle cross-chain mint budgets without operator consent	Acknowledged
[L-5] Tokens can be sent cross-chain to blocked users on non-EVM chains	Acknowledged
[I-01] Use named imports	Resolved
[I-02] Use capital case for constants	Resolved
[I-03] Use Ownable2StepUpgradeable instead of OwnableUpgradeable	Acknowledged
[I-04] Use _checkZeroAddress in DelayedUpgradeable::requestUpgradeToAndCall	Resolved
[I-05] Emit missing events in state-changing functions	Acknowledged
[I-06] In Solidity don't initialize to default values	Resolved
[I-07] No restriction on callers to BullionMinter::requestToMint, requestToRedeem allows off-chain parameter checks to be bypassed	Acknowledged

[I-08] Owner can rug-pull user tokens by front-running calls to <code>BullionMinter::requestToMint</code> , <code>requestToRedeem</code> to change <code>poolAccountA</code> , <code>poolAccountB</code>	Acknowledged
[I-09] Remove unused variables	Acknowledged
[I-10] <code>MToken::mintTo</code> nonce never enforced on-chain	Acknowledged
[I-11] Use named mappings to explicitly denote the purpose of keys and values	Resolved
[I-12] Remove obsolete <code>return</code> statement when already using named return variables	Resolved
[I-13] Fallback oracle contains no staleness check	Acknowledged
[I-14] Consider using <code>SafeCast</code> instead of unsafe downcasting	Resolved
[I-15] No way to specifically pause CCIP send transactions as exists for <code>LayerZero</code> send transactions	Acknowledged
[I-16] Cross-chain transfers can cause <code>totalSupply</code> to be greater than <code>mint-Budget</code> on the destination chain	Acknowledged
[G-1] Cache storage to prevent identical storage reads	Resolved
[G-2] Use <code>calldata</code> instead of <code>memory</code> for read-only inputs	Resolved
[G-3] Perform <code>dstAddrLen</code> revert checks first in <code>MTokenMessengerLZ::lzSendTokenToChain</code>	Resolved

7 Findings

7.1 Low Risk

7.1.1 Existing `MToken::revoker` can prevent owner from setting new revoker

Description: The owner uses `MToken::setRevoker` to:

- first request a new revoker change by setting `nextRevoker`, `etNextRevoker`
- second complete the new revoker change as long as `etNextRevoker != 0 && etNextRevoker < block.timestamp`

However the current revoker can always call `MToken::revokeNextRevoker` to set `etNextRevoker = 0` preventing the second step.

Recommended Mitigation: `MToken::revokeNextRevoker` should have the `onlyOwner` modifier instead of `onlyRevoker`:

```
- function revokeNextRevoker() public onlyRevoker {  
+ function revokeNextRevoker() public onlyOwner {
```

The `Delayable` contract which has the same function does exactly this:

```
function revokeNextRevoker() public onlyOwner {  
    etNextRevoker = 0;  
}
```

Matrixdock: Fixed in commit [ab3726b](#).

Cyfrin: Verified.

7.1.2 `MToken::mintTo`, `redeem`, `ccReceiveToken`, `pack`, `unpack` never enforce the blocklist

Description: `MToken` contains blocklist functionality however `mintTo`, `redeem`, `ccReceiveToken`, `pack`, `unpack` never check the blocklist allowing minting and redemptions for blocked users

Recommended Mitigation: * `mintTo`, `ccReceiveToken` should call `_checkBlocked(receiver)`

- `redeem` should call `_checkBlocked(customer)`
- `pack`, `unpack` should call `_checkBlocked(tokenOwner)`

Matrixdock: Acknowledged; `mintTo`, `redeem`, `pack` are privileged operations, and access to them is enforced by the operator. We will carefully evaluate other scenarios in future updates. For now, the current implementation will remain unchanged.

7.1.3 `MToken::transfer`, `transferFrom`, `multiTransfer` allow transferring to blocked recipients

Description: `MToken::transfer`, `transferFrom`, `multiTransfer` only verify that the sender is not blocked, but allow transferring to blocked recipients.

Recommended Mitigation: In `MToken::transfer`, `transferFrom` add a call to `_checkBlocked(_recipient)` to prevent transfers to blocked recipients.

Matrixdock: Acknowledged; this is the expected behavior; we only block the “sender” address.

7.1.4 Using `tx.origin` allows possibility for third-party contract to shuffle cross-chain mint budgets without operator consent

Description: `MToken::ccSendMintBudget`:

- has modifier `onlyMessenger` to enforce that only the approved messenger contract can directly call this function

- uses `_checkOperator(tx.origin)` to enforce that the transaction was initiated by an operator

Impact: Third-party smart contracts that the operator interacts with can shuffle `mintBudget` between legitimate chains without operator consent.

Proof of Concept: The usage of `tx.origin` opens up the following potential attack path:

- 1) Operator calls `MaliciousContract::innocentFunction`
- 2) `MaliciousContract` calls `MTokenMessengerLZ::lzSendMintBudgetToChain` which is external with no access control
- 3) This calls `MToken::ccSendMintBudget` and both checks pass (`onlyMessenger` and `_checkOperator(tx.origin)`)

Recommended Mitigation: Access control should be enforced at the entry point where `msg.sender` is the actual external caller, not deep in the call stack using `tx.origin`. Hence:

- remove `_checkOperator(tx.origin)` from `MToken::ccSendMintBudget`
- add and use a similar function `_checkOperator(msg.sender)` to `MTokenMessengerLZ::lzSendMintBudgetToChain` and `MTokenMessenger::sendMintBudgetToChain`. This function could just call `MToken::operator` to get the operator then compare to `msg.sender` so the implementation is simple

Matrixdock: Acknowledged; we consider the risk to be manageable, and in order to preserve sufficient flexibility, we have decided to keep the current implementation for now.

7.1.5 Tokens can be sent cross-chain to blocked users on non-EVM chains

Description: The purpose of the upgrade is to allow cross-chain transfers to non-EVM chains. However blocked users are still stored using address which doesn't support non-EVM chains:

```
mapping(address => bool) public isBlocked;
```

Additionally `MTokens::msgOfCcSendToken` checks whether the receiver is blocked only when `receiverBytes` is 20 bytes (EVM-compatible chains). However the address length can vary across non-EVM chains so `MTokens` deployed on EVM chains can't prevent cross-chain transfers to blocked accounts on non-EVM chains.

```
function msgOfCcSendToken(
    address sender,
    bytes calldata receiverBytes,
    uint256 value
) public view returns (bytes memory message) {
    _checkBlocked(sender);
    if (receiverBytes.length == 20) {
        address receiver = address(bytes20(receiverBytes));
        _checkBlocked(receiver);
    }
    ...
}
```

Recommended Mitigation: Consider updating the block list to not be limited to only EVM addresses (20 bytes); this requires storing blocked addresses as `bytes` or more efficiently storing their hash as `bytes32`.

Matrixdock: Acknowledged; for transfer-related methods within the chain, we only check the sender address. Similarly, for cross-chain transfers, we also only check the sender address. We then expect the address on the destination chain to already be blocked.

7.2 Informational

7.2.1 Use named imports

Description: The codebase already uses named imports in some files, for example in `MTokenMessengerLZ.sol`. Using named imports is best practice and ideally should be used everywhere else:

```
MToken.sol
4:import "@openzeppelin/contracts-upgradeable/token/ERC20/extensions/ERC20PermitUpgradeable.sol";
5:import "../DelayedUpgradeable.sol";
6:import "../interfaces/ICCCClient.sol";

BullionMinter.sol
4:import "@openzeppelin/contracts/token/ERC20/IERC20.sol";
5:import "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
6:import "../DelayedUpgradeable.sol";

DelayedUpgradeable.sol
4:import "@openzeppelin/contracts/proxy/utils/UUPSUpgradeable.sol";
5:import "@openzeppelin/contracts-upgradeable/access/OwnableUpgradeable.sol";

Delayable.sol
4:import "../DelayedUpgradeable.sol";
```

Matrixdock: Fixed in commit [19002d2](#).

Cyfrin: Verified.

7.2.2 Use capital case for constants

Description: The codebase already uses capital case for constants, for example in `Delayable.sol`. Using capital case for constants is good practice and should be done consistently:

```
MToken.sol
64: uint constant TagSendToken = 2;
65: uint constant TagSendMintBudget = 3;

BullionMinter.sol
15: uint8 constant public prepriceDecimal = 6;
16: uint8 constant public slippageDecimal = 6;
17: uint constant public delayMax = 59;
```

Matrixdock: Fixed in commit [039612b](#).

Cyfrin: Verified.

7.2.3 Use `Ownable2StepUpgradeable` instead of `OwnableUpgradeable`

Description: Use [Ownable2StepUpgradeable](#) instead of `OwnableUpgradeable`:

```
DelayedUpgradeable.sol
5:import "@openzeppelin/contracts-upgradeable/access/OwnableUpgradeable.sol";
7:abstract contract DelayedUpgradeable is OwnableUpgradeable, UUPSUpgradeable {
```

Matrixdock: Acknowledged; this change is a bit cumbersome to make. For example, LayerZero's `OAppCoreUpgradeable` inherits from `OwnableUpgradeable`, which causes compilation errors in `MTokenMessengerLZ`. Considering that many of our contracts inherit from `DelayedUpgradeable`, the impact of this change would be quite large, so we've decided to postpone it for now.

7.2.4 Use `_checkZeroAddress` in `DelayedUpgradeable::requestUpgradeToAndCall`

Description: `DelayedUpgradeable` has an internal function `_checkZeroAddress` which reverts if an input is the zero address, but this isn't used anywhere by `DelayedUpgradeable`.

Ideally it would be used inside `DelayedUpgradeable::requestUpgradeToAndCall` to prevent requesting an upgrade to the zero address.

There are a number of other contracts which inherit from `DelayedUpgradeable`; consider using `_checkZeroAddress` in those contracts where functions set important address variables.

Matrixdock: Fixed in commit [ce3d7d5](#).

Cyfrin: Verified.

7.2.5 Emit missing events in state-changing functions

Description: Emit missing events in state-changing functions:

- `Delayable::revokeNextUpgrade`, `revokeNextDelay`, `revokeNextRevoker`, `revokeNextOperator`
- `MToken::setDisableCcSend`, `setNFTContract`, `revokeNextDelay`, `revokeNextOperator`, `revokeNextMessenger`, `revokeNextRevoker`, `revokeNextUpgrade`
- `MTokenMessengerBaseUpgradeable::revokeNextUpgrade`

Matrixdock: Acknowledged; will add these events in future updates.

7.2.6 In Solidity don't initialize to default values

Description: In Solidity don't initialize to default values:

```
BullionMinter.sol
29:         for(uint i=0; i<_tokensAcceptedByA.length; i++) {
32:         for(uint i=0; i<_tokensAcceptedByB.length; i++) {
```

Matrixdock: Fixed in commit [f7c8b45](#).

Cyfrin: Verified.

7.2.7 No restriction on callers to `BullionMinter::requestToMint`, `requestToRedeem` allows off-chain parameter checks to be bypassed

Description: `BullionMinter::requestToMint`, `requestToRedeem` have comments stating that:

```
// Most parameters are not checked here and are handled by the off-chain service
```

However there are no restrictions on who can call these functions, meaning that they can be called directly by-passing parameter checks with occur in the off-chain services.

Recommended Mitigation: Enforce that only "operator" addresses controlled by the off-chain services can call these functions.

Matrixdock: Acknowledged; the off-chain program will process the transactions that call these two on-chain methods, and from a business perspective, these two methods are intended to be publicly accessible.

7.2.8 Owner can rug-pull user tokens by front-running calls to `BullionMinter::requestToMint`, `requestToRedeem` to change `poolAccountA`, `poolAccountB`

Description: When users call `BullionMinter::requestToMint`, `requestToRedeem` their tokens are always sent to `poolAccountA`, `poolAccountB`.

However the owner can front-run user calls by calling `setPoolAccountA`, `setPoolAccountB` to change these addresses, causing the user tokens to go to an address users did not expect.

Recommended Mitigation: The simplest fix is to allow callers to pass input parameters `expectedPoolAccountA`, `expectedPoolAccountB` to `BullionMinter::requestToMint`, `requestToRedeem`, then inside those functions verify that the expected inputs match the actual storage values.

Another option is to change `setPoolAccountA`, `setPoolAccountB` to have a time-delay scheme where a "proposed" new value is first set, then some time has to expire before the proposed value becomes the new value.

Matrixdock: Acknowledged; our owner is securely held in the wallet's escrow system. Every transaction undergoes a two-step process of signature and verification. We trust the owner's actions.

7.2.9 Remove unused variables

Description: Remove unused variables:

```
BullionMinter.sol
15:     uint8 constant public prepriceDecimal = 6;
16:     uint8 constant public slippageDecimal = 6;
```

Matrixdock: Acknowledged; these variables were retained in the code for historical reasons, making it easier to view these two values.

7.2.10 MToken::mintTo nonce never enforced on-chain

Description: `MToken::mintTo` takes `nonce` as an input parameter but the `nonce` itself is never recorded as being used on-chain, meaning that the same `nonce` can be used multiple times.

Recommended Mitigation: Consider using a mapping to invalidate a `nonce` once it has been used.

Matrixdock: Acknowledged; the `nonce` is used off-chain only. Uniqueness of the `nonce` is enforced by our off-chain system, so no on-chain validation is performed here and the current implementation is kept as is.

7.2.11 Use named mappings to explicitly denote the purpose of keys and values

Description: Most of the codebase already uses named mappings to explicitly denote the purpose of keys and values, except:

```
MToken.sol
40:     mapping(address => bool) public isBlocked;
```

Matrixdock: Fixed in commit [24acb1f](#).

Cyfrin: Verified.

7.2.12 Remove obsolete return statement when already using named return variables

Description: Remove obsolete `return` statement when already using named return variables:

- `MToken::ccSendMintBudget`
- `MTokenMessenger::sendDataToChain`

Matrixdock: Fixed in commit [7527952](#).

Cyfrin: Verified.

7.2.13 Fallback oracle contains no staleness check

Description: Fallback oracle contains no staleness check in `MTokenMain::increaseMintBudget`:

```
function increaseMintBudget(uint112 mintBudgetDelta) public onlyOperator {
    uint _usedReserve = usedReserve + mintBudgetDelta;
    // prettier-ignore
```

```

(
    /*uint80 roundID*/,
    int reserveFromFeed,
    /*uint startedAt*/,
    uint updatedAt,
    /*uint80 answeredInRound*/
) = AggregatorV3Interface(reserveFeed).latestRoundData();

if (updatedAt + ORACLE_OFFLINE_THRESHOLD < block.timestamp) {
    // use fallback reserve feed
    // @audit no staleness check after using fallback oracle?
    (, reserveFromFeed, , , ) = AggregatorV3Interface(fallbackFeed)
        .latestRoundData();
}

if (int(_usedReserve) > reserveFromFeed) {
    revert ReserveNotEnough(reserveFromFeed, int(_usedReserve));
}
mintBudget += uint112(mintBudgetDelta);
usedReserve = uint112(_usedReserve);
}

```

Matrixdock: Acknowledged; the fallback feed does not require strong timeliness guarantees. As a result, a staleness check is not implemented, and the current behavior is intentional.

7.2.14 Consider using SafeCast instead of unsafe downcasting

Description: Consider using [SafeCast](#) instead of unsafe downcasting:

```

MToken.sol
323:     mintBudget = uint112(mintBudget - amount);
336:     mintBudget += uint112(amount);
428:     value = uint112(convertToSharedDecimals(value));
462:     value = uint112(convertToLocalDecimals(value));

MTokenMain.sol
93:     mintBudget += uint112(mintBudgetDelta);
94:     usedReserve = uint112(_usedReserve);
100:    usedReserve = uint112(_usedReserve);

```

Impact: For example in `MToken::redeem` if an unsafe downcast occurs, `mindBudget` will be reduced by a much smaller amount than was actually burned breaking the internal accounting.

Matrixdock: Fixed in commit [26e89fa](#).

Cyfrin: Verified.

7.2.15 No way to specifically pause CCIP send transactions as exists for LayerZero send transactions

Description: The LayerZero "send" functions `MTokenMessengerLZ::lzSendTokenToChain`, `lzSendMintBudgetToChain` have modifier `onlyLZNotPaused` which allows the owner to pause LayerZero "send" transactions.

However the equivalent CCIP "send" functions `MTokenMessenger::sendTokenToChain`, `sendMintBudgetToChain` are missing a similar modifier meaning only CCIP "send" transactions are unable to be paused by the owner.

Recommended Mitigation: Add a `onlyCCIPNotPaused` modifier to `MTokenMessenger::sendTokenToChain`, `sendMintBudgetToChain` allowing the owner to pause only CCIP "send" transactions, similar to what the owner can already do for LayerZero "send" transactions.

Matrixdock: Acknowledged; this is a historical issue. Initially, we designed the system to use CCIP for cross-chain messaging, but later we decided to gradually migrate to LayerZero. In `MToken`, there is a `disableCcSend` field that

can pause all cross-chain sending. Since we are in the process of gradually deprecating CCIP, we will leave this unchanged for now.

7.2.16 Cross-chain transfers can cause `totalSupply` to be greater than `mintBudget` on the destination chain

Description: `MToken::ccReceiveToken` (used by both LayerZero & CCIP) mints tokens without checking the destination chain's `mintBudget`, while `MToken::mintTo` enforces the `mintBudget`.

Impact: A chain's `totalSupply` can exceed its `mintBudget` through incoming cross-chain transfers.

Recommended Mitigation: If this is not intended, then cross-chain transfers should:

- increase `mintBudget` on the source chain
- decrease `mintBudget` on the destination chain

Matrixdock: Acknowledged; this is the expected behavior.

7.3 Gas Optimization

7.3.1 Cache storage to prevent identical storage reads

Description: Cache storage to prevent identical storage reads:

```
Delayable.sol
// cache `delay` and value assigned to `etNextDelay` in `setDelay`
// then use cached values when emitting event `SetDelayRequest`
76:         etNextDelay = uint64(block.timestamp) + delay;
77:         emit SetDelayRequest(delay, _delay, etNextDelay);

// cache value assigned to `etNextRevoker` in `setRevoker`
93:         etNextRevoker = uint64(block.timestamp) + delay;
94:         emit SetRevokerRequest(revoker, newRevoker, etNextRevoker);

// cache value assigned to `etNextOperator` in `setOperator`
110:        etNextOperator = uint64(block.timestamp) + delay;
111:        emit SetOperatorRequest(operator, newOperator, etNextOperator);

MToken.sol - `setDelay, setMessenger, setRevoker, setOperator` - similar to above

MTokenMessengerBaseUpgradeable.sol - `setDelay` - same as above

BullionMinter.sol
// cache `poolAccountA` in `requestToMint`
84:         IERC20(transferredToken).safeTransferFrom(msg.sender, poolAccountA, amount);
85:         emit MintRequest(transferredToken, forToken, msg.sender, poolAccountA, amount, preprice,
↪      slippage, extraData);

// cache `poolAccountB` in `requestToRedeem`
92:         IERC20(transferredToken).safeTransferFrom(msg.sender, poolAccountB, amount);
93:         emit RedeemRequest(transferredToken, forToken, msg.sender, poolAccountB, amount, preprice,
↪      slippage, extraData);
```

Matrixdock: Fixed in commit [25ca660](#).

Cyfrin: Verified.

7.3.2 Use calldata instead of memory for read-only inputs

Description: Use calldata instead of memory for read-only inputs where those inputs are never passed into functions requiring them as memory:

```
BullionMinter.sol
23:         address[] memory _tokensAcceptedByA,
24:         address[] memory _tokensAcceptedByB
41:         address[] memory _tokensAcceptedByA,
42:         address[] memory _tokensAcceptedByB

MToken.sol
362:        address[] memory _recipients,
363:        uint256[] memory _values
```

Matrixdock: Fixed in commit [2c3d6bc](#).

Cyfrin: Verified.

7.3.3 Perform dstAddrLen revert checks first in MTokenMessengerLZ::lzSendTokenToChain

Description: In MTokenMessengerLZ::lzSendTokenToChain it is more efficient to perform the revert checks related to dstAddrLen first; if the transaction is going to revert because of an incorrect length, want to avoid doing all the

work in `ccSendToken` and `sendThroughLZ`.

Matrixdock: Fixed in commit [a0cb8ca](#).

Cyfrin: Verified.